Advanced Python Programming

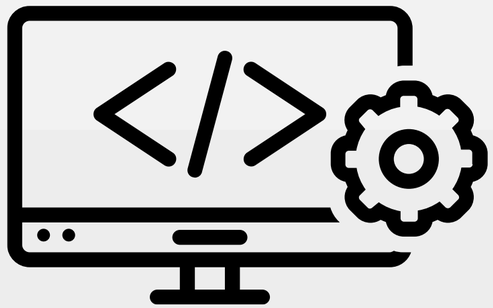
List Comprehensions

```
[ x for x in range(10) ]
```


by

Prof. M. Iqbal Bhat
Govt. Degree College Beerwah

TABLE OF CONTENT



WHAT ARE COMPREHENSIONS?

- In Python, comprehensions are concise and expressive ways to create data structures like lists, sets, and dictionaries.
- Comprehensions make your code more readable and efficient by reducing the need for explicit loops.
- Comprehensions allow to create a new list from an existing list by applying a filter and/or transformation to each element of the existing list.

BENEFITS OF COMPREHENSIONS?

- Comprehensions are more readable and concise than traditional loops.
- They are often faster and more efficient.
- They encourage a more functional programming style.

LIST COMPREHENSION

- List comprehensions allow you to create lists in a more compact and readable way.

Syntax

Python

```
new_list = [expression for element in iterable if condition]
```

Use code with caution. Learn more

- `new_list` : The new list.
- `expression` : The expression to be evaluated for each element of the iterable.
- `iterable` : The iterable to be processed.
- `condition` : An optional condition that must be met for the element to be included in the new list.

LIST COMPREHENSION

```
new_list = [expression for element in iterable if condition]
```

numbers = [1, 2, 3, 4, 5]

squares = [x**2 for x in numbers]

Create a list of all even numbers from 1 to 10.

even_numbers = [number for number in range(1, 11) if number % 2 == 0]

LIST COMPREHENSION

```
new_list = [expression for element in iterable if condition]
```

- Create a list of all the possible combinations of two numbers from a list of numbers.

```
numbers = [1, 2, 3, 4]
combinations = [(number1, number2) for number1 in numbers for number2 in numbers if number1 != number2]
```

SET COMPREHENSION

- Set comprehensions are similar to list comprehensions but create sets.

Syntax

```python
new_set = {expression for element in iterable if condition}
```

Use code with caution. Learn more

- `new_set` : The new set.
- `expression` : The expression to be evaluated for each element of the iterable.
- `iterable` : The iterable to be processed.
- `condition` : An optional condition that must be met for the element to be included in the new set.

SET COMPREHENSION

```
new_set = {expression for element in iterable if condition}
```

numbers = [1, 2, 2, 3, 3, 4, 5]

unique_numbers = {x for x in numbers}

Create a set of all even numbers from 1 to 10.

even_numbers = {number for number in range(1, 11) if number % 2 == 0}

SET COMPREHENSION

```
new_set = {expression for element in iterable if condition}
```

- Create a set of all the unique vowels in a string.

string = "This is a test string."

vowels = {vowel for vowel in string if vowel in "aeiou"}

DICTIONARY COMPREHENSION

- Dictionary comprehensions allow you to create dictionaries in a concise manner.

Syntax

Python

```
new_dict = {key: expression for element in iterable if condition}
```

Use code with caution. Learn more

- `new_dict` : The new dictionary.
- `key` : The key for each element in the new dictionary.
- `expression` : The expression to be evaluated for each element of the iterable.
- `iterable` : The iterable to be processed.
- `condition` : An optional condition that must be met for the element to be included in the new dictionary.

DICTIONARY COMPREHENSION

```
new_dict = {key: expression for element in iterable if condition}
```

names = ["Alice", "Bob", "Charlie"]
name_lengths = {name: len(name) for name in names}

Create a dictionary of all even numbers from 1 to 10, with the key being the number and the value being the square of the number.

even_numbers = {number: number ** 2 for number in range(1, 11) if number % 2 == 0}

DICTIONARY COMPREHENSION

```
new_dict = {key: expression for element in iterable if condition}
```

- Create a dictionary of all the possible mappings from a set of keys to a set of values.

keys = {1, 2, 3}

values = {"a", "b", "c"}

mappings = {key: value for key in keys for value in values}

List Comprehensions with single if

```
new_dict = {key: expression for element in iterable if condition}
```

We can include a single if condition to filter elements from an iterable and include only those that meet the condition

[expression for item in iterable if condition]

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = [x for x in numbers if x % 2 == 0]

LIST COMPREHENSION WITH if-else CONDITION

We can use if-else conditions in list comprehensions to specify different expressions for inclusion based on a condition.

[expression_if_true if condition else expression_if_false for item in iterable]

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_odd_labels = ["Even" if x % 2 == 0 else "Odd" for x in numbers]
```

```
scores = [75, 88, 92, 45, 60, 78, 89, 55, 94]
pass_fail = ["Pass" if score >= 70 else "Fail" for score in scores]
```

List Comprehensions with Multiple if Conditions:

[expression_if_true if condition else expression_if_true if condition else expression_if_true condition else expression_if_false for item in iterable]


```
grades = [88, 75, 92, 60, 78, 89, 55, 94]
letter_grades = ["A" if grade >= 90 else "B" if grade >= 80 else "C" if grade >= 70 else "D" if grade >= 60 else "F" for grade in grades]
```

List Comprehensions with Multiple if Conditions:

We can use multiple if conditions in list comprehensions by chaining them with and or or operator

[expression for item in iterable if condition1 and condition2]

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
divisible_by_2_and_3 = [x for x in numbers if x % 2 == 0
                        and x % 3 == 0]
```

Nested Comprehensions:

Nested Comprehensions:

- Nested comprehensions in Python allow you to create more complex data structures by embedding one or more comprehensions within another.
- They are a powerful feature that can make your code more concise and expressive.

Nested Comprehensions:

[[expression for item in inner_iterable] for item in outer_iterable]

- The outer comprehension iterates over an outer iterable (e.g., a list or range).
- For each item in the outer iterable, there's an inner comprehension that iterates over an inner iterable.
- The expression within the inner comprehension defines how the inner elements are generated.

Zip function

Nested Comprehensions:

[[expression for item in inner_iterable] for item in outer_iterable]

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened_matrix = [element for row in matrix for element in row]
```

Nested Comprehensions:

[[expression for item in inner_iterable] for item in outer_iterable]

```
matrix1 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
matrix2 = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]
result_matrix = [
                  [matrix1[i][j] + matrix2[i][j] for j in
                  range(len(matrix1[0]))]
                  for i in range(len(matrix1))
                 ]
```

Zip function:

The zip() function in Python is used to combine multiple iterable objects (like lists or tuples) into a single iterable, effectively creating pairs of corresponding elements. It's a versatile function and can be used in various scenarios.


```
names = [“Muhammad", “Ahmed", “Hamza"]
scores = [98, 92, 78]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

Zip function:

Finding the Maximum Element in Multiple Lists

```
keys = ["name", "age", "city"]
values = ["Alice", 30, "New York"]

user_info = dict(zip(keys, values))
```

Finding the Maximum Element in Multiple Lists

```
list1 = [34, 56, 78]
list2 = [12, 89, 45]

maximum_values = [max(pair) for pair in zip(list1, list2)]
```

Zip function:

Pairwise Sum of Lists

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]


sums = [x + y for x, y in zip(list1, list2)]
```

Combining Two Lists into a Dictionary

```
keys = ['name', 'age', 'city']
values = ['Alice', 30, 'New York']


user_info = {key: value for key, value in zip(keys, values)}
```

Zip function:

Unzipping into Separate Lists:

Suppose you have a list of pairs, and you want to separate them into two lists, one for each element of the pair. You can use zip() and the * (unpacking) operator to accomplish this

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
# Unzip into separate lists
numbers, letters = zip(*pairs)


data = [(1, 'A', 'X'), (2, 'B', 'Y'), (3, 'C', 'Z')]
# Unzip into separate lists
numbers, letters, symbols = zip(*data)
```

THANK YOU